Fast Inference of Therapeutic PROTEIN–Ligand Interactions (F.I.T. Proteins)

Alfonso Payán[1], Ricardo Pérez[1], and Pablo Pérez[1]

[1]IE University

December 2025

Abstract

Cancer fundamentally arises from the uncontrolled growth of cells, driven by dysregulated molecular signals that sustain proliferation. Predicting BRD4 binders is like trying to cut the power cable feeding a rapidly expanding tumor: BRD4 functions as a transcriptional switchboard that keeps cancer cells dividing, and only a few molecules in a vast chemical universe can truly shut it down. In this project, we use machine learning models to identify those rare compounds by learning the subtle structural patterns that drive protein–ligand binding. By analyzing DNA-encoded library (DEL) molecules through fingerprints, SMILES-based models, and graph neural networks, our approach seeks to highlight which small molecules are most likely to disrupt BRD4's growth-promoting activity, providing a computational pathway toward more efficient early-stage drug discovery.

1 Introduction

1.1 Background and motivation

Drug discovery is slow and takes a lot of time. However, due to andvancements in technology, now we are able to use machine learning i the crucial step of virtual screening, reducing valuable time and resources in the process. (Zhong et al.). BRD4, a member of the bromodomain and extraterminal (BET) family, is a protein that regulates various cellular processes, controlling cancers, inflamations and others. Its selective inhibition has shown high promise in oncology, taking scientists one step ahead on the discovery of a potential cure for some cancers. (Dong and Hao; Jia et al.)

DNA-encoded library (DEL) technology addresses the need to need to screen a large space of molecules, synthesizing in pooled format. (Yuen and Franzini; Favalli et al.). They use next-generation sequencing of retained bar codes, revealing what chemotypes interact with the target protein. However, the compounds are assembled combinatorially from a limited set of building blocks, which can be easily captured by a complex machine learning method.

The Big Encoded Library for Chemical Assessment (BELKA) is a DEL dataset generated by Leash Biosciences and released as part of the NeurIPS 2024 Kaggle competition "Predict New Medicines with BELKA," which focuses on predicting small-molecule binding to three protein targets, one of them being BRD4. (Blevins et al.). In this project we work with a stratified subset of 50 000 BRD4-targeting molecules, out of a dataset of 133 million, that preserves the original active rate of roughly 0.5%.

From a machine learning perspective, this setting is challenging in two ways. First, the extreme class imbalance makes naive metrics such as accuracy uninformative and pushes us toward other metrics that reward noticing rate activities. Second, the combinatorial construction of BELKA means that the random splits will leak information that complex models will notice right away. This will result in an inflated performance when validating and testing, since the models will memorize the building block structure and not the chemical properties. In consequence, the experiment will diverge from real settings, where building blocks and libraries are unseen during training (Blevins et al.).

For the design choice, we needed to consider different scenarios, since different representations are required for different learning methods. In Classical Informatics, they use fixed fingerprints such as extended-connectivity fingerprints (ECFP), which encode local atom environments as high-dimensional binary vectors(David et al.). However, when using Neural Networks, which learn task-specific embeddings directly from structure, the usual thing to do is tokenizing SMILES or turning the molecule into a graph. SMILES (Simplified Molecular Input Line Entry System) represents a molecule as a one-dimensional string that records atoms, bonds, branches and ring closures, while implicitly encoding the underlying molecular graph (Weininger, "SMILES, a Chemical Language").

However, they're not unique, since the same molecule may have different SMILES representations (David et al.). Figure 1 illustrates how a single compound can be viewed as a 3D structure, a 2D molecular graph and a SMILES string, adapted from Xiao *et al.*

1.2 Problem formulation

Since we want to see which molecule binds or not, we cast BRD4 binding prediction as a supervised binary classification task. Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ denote the dataset, where x_i denotes a small molecule and $y_i \in \{0, 1\}$ indicates whether it is labeled as a BRD4 binder

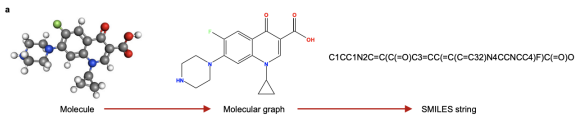


Figure 1: Illustration of a molecule, its 2D molecular graph and the corresponding SMILES string (adapted from Xiao *et al.*).

($y_i = 1$) or non-binder ($y_i = 0$). As explained previously, each molecule can be represented in several ways:

- as a SMILES string $s_i = (t_{i,1}, \ldots, t_{i,L_i})$ over a token vocabulary;
- as an ECFP fingerprint $\phi_{\text{ECFP}}(x_i) \in \{0,1\}^d$, obtained by hashing circular atom environments;
- as a molecular graph $G_i = (V_i, E_i, \mathbf{X}_i, \mathbf{E}_i)$, where V_i and E_i are the sets of atoms and bonds, and $\mathbf{X}_i, \mathbf{E}_i$ denote node- and edge-level feature matrices.

The main task of our model is to fit a parametric model f_θ that maps one of these representations to a real-valued score $f_\theta(x_i) \in \mathbb{R}$, interpreted as the probability of BRD4 binding, such that the ranking achieves high precision under extreme circumstances of imbalance.

Due to the nature of our data, we needed to enforce out-of-distribution (OOD) evaluation with respect to the DEL building blocks. For this, we exploit the combinatorial structure of the library. Each BELKA molecule is assembled from three building blocks attached to a shared core. What we did is to make a partition of the full set of building blocks into 3 pools: train, validation and test, each disjoint from each other, and constructed 3 splits:

- $\mathcal{D}_{\text{train-in}}$: molecules using only train blocks;
- $\mathcal{D}_{\text{val-ood}}$: molecules with at least one validation-only block;
- $\mathcal{D}_{\text{test-ood}}$: molecules with at least one test-only block or belonging to a held-out library.

Our models were trained only on the data contained in $\mathcal{D}_{\text{train-in}}$, hyperparameters were selected using $\mathcal{D}_{\text{val-ood}}$, and final metrics were reported on $\mathcal{D}_{\text{test-ood}}$. We only tested once.

As our primary evaluation metric, we use average precision (AP), but we also capture ROC-AUC as a secondary measure (Blevins et al.).

Within this setup, we focus on two main questions:

1. How well do classical tree-based models perform under a block-aware OOD split, when evaluated by AP and ROC-AUC on $\mathcal{D}_{\text{val-ood}}$ and $\mathcal{D}_{\text{test-ood}}$?
2. Do learned representations based on convolutional neural networks (CNNs) and graph neural networks (GNNs) provide gains in OOD AP under the same evaluation protocol, or do small data sets, and severe imbalance offset their potential benefits?

2 Data and Preprocessing

2.1 Dataset description

The data used for this project comes *NeurIPS 2024 – Predict New Medicines with BELKA*, a Kaggle competition. The data provided included binary binding labels for more than 133 million small molecules generated by DEL technology and screened against three protein targets, one of which was BRD4. All this molecules as well as the three underlying building block SMILES were present (Blevins et al.).

Due to the capacity of our hardware, we restrict our attention to a stratified subset of 50 000 molecules extracted from the competition training data. We stratified the subset by ourselves, preserving the original class imbalance of approximately 0.5% binders by design, resulting in 269 positives and 49 731 negatives (a final positive rate of 0.54%).

2.2 Chemistry preprocessing

The Kaggle molecules in SMILES format contained a triazine core and a special token representing the DEL DNA linker ("[Dy]") together with three separate building-block SMILES (Blevins et al.). This doesn't work for our project, since we need our models to focus on the underlying chemistry behind the molecules rather than on the structure of DEL library molecules. For machine learning purposes we needed to perform preprocessing steps:

Linker neutralization. The token ([Dy]) was replaced by a neutral atom symbol, while preserving SMILES structure. This keeps the small-molecule scaffold intact but removes unnecessary information which is not expected to drive BRD4 binding in a transferable way.

SMILES canonicalization. As we explained previously, each molecule may have different SMILES representations. That's why for all full-molecule and building-block SMILES, we applied a process where we converted them to RDKit molecules and then re-exported them in canonical form. Canonicalization ensures that the same molecules is mapped to a unique string, which is crucial for de-duplication (Weininger, "SMILES. 2").

Data cleaning. During pre processing, we will discard the molecules which RDKit fails to parse, the rows with missing binding labels, and the molecules that were duplicated but in different SMILES form. These steps slightly reduce the total sample size and variance, but avoid training our models on ill-defined, inconsistent or duplicated entries.

2.3 Block-aware split

DEL chemistry is modular from construction: every molecule will be assembled from three building blocks attached to a share core (Blevins et al.). The combinatorial design nature of the molecules results in shared

fragments among them, even if their structures differ. A completely random split would leak a lot of information across the train-validation-test sets and inflate the models' overall performance. Because our models need to be able to perform nicely on new chemistry, they need to learn the chemical patterns across the molecules and not the DEL structure, that's why we construct a block-aware split, this is how we did it. Let $\mathcal{B}$ denote the set of unique building blocks appearing in the 50k BRD4 subset, and let $B(x_i) \subseteq \mathcal{B}$ be the three building blocks used to synthesize molecule x_i. We created a partition of the building blocks on 3 disjoint subsets:

$$\mathcal{B} = \mathcal{B}_{\text{train}} \cup \mathcal{B}_{\text{val}} \cup \mathcal{B}_{\text{test}}, \qquad \mathcal{B}_{\text{train}} \cap \mathcal{B}_{\text{val}} \cap \mathcal{B}_{\text{test}} = \varnothing,$$

and define molecule-level splits as

$$\begin{aligned} \mathcal{D}_{\text{train-in}} &= \{(x_i, y_i) : B(x_i) \subseteq \mathcal{B}_{\text{train}}\}, \\ \mathcal{D}_{\text{val-ood}} &= \big\{(x_i, y_i) : B(x_i) \cap \mathcal{B}_{\text{val}} \neq \varnothing, \big\}, \\ &\qquad B(x_i) \cap \mathcal{B}_{\text{test}} = \varnothing \\ \mathcal{D}_{\text{test-ood}} &= \{(x_i, y_i) : B(x_i) \cap \mathcal{B}_{\text{test}} \neq \varnothing\}. \end{aligned}$$

This results in OOD samples, because any of the building blocks will be present in one of the subsets, so molecules in $\mathcal{D}_{\text{val-ood}}$ and $\mathcal{D}_{\text{test-ood}}$ will always contain some information that is never observed during the training stage. This supports the recommendations made for QSAR benchmarks, were scaffold-based splits are commonly recommended over random splits (David et al.). Moreover, it approximates reality, where virtual screening processes are made upon new data. In our implementation this yields 34 819 train-in molecules, 7 268 validation-OOD molecules, and 7 913 test-OOD molecules, with positive rates of 0.34%, 0.41%, and 1.52%, respectively (Table 1).

Table 1: Block-aware splits for the BRD4 50k subset.

Split	# molecules	# positives	Pos. rate (%)
$\mathcal{D}_{\text{train-in}}$	34 819	119	0.34
$\mathcal{D}_{\text{val-ood}}$	7 268	30	0.41
$\mathcal{D}_{\text{test-ood}}$	7 913	120	1.52

2.4 Feature representations

For our project, we will have 3 different types of representation families that map SMILES molecules into fixed-dimensional feature spaces, which will allow us to perform our experiments. These are: handcrafted circular fingerprints (ECFP), tokenized SMILES strings, and molecular graphs.

ECFP fingerprints. Extended-connectivity fingerprints (ECFPs) are circular topological fingerprints that encode local atom environments up to a given radius into a fixed-length bit vector (Rogers and Hahn). For each molecule x_i with graph G_i, we map atom neighborhoods of radius r into integer identifiers $h_k(v)$, and then map this identifiers into a binary vector via a modulo operation. This yields

$$\phi_{\text{ECFP}}(x_i) \in \{0,1\}^d,$$

where bit j indicates the presence of at least one substructure whose hash maps to position j. For our experiments, we use a standard configuration widely used, which is a radius $r = 2$, and a vector of $d = 2048$ bits (David et al.).

SMILES token sequences. The Simplified Molecular-Input Line-Entry System (SMILES) represents a molecule as a linearized string over an alphabet of atom symbols, bond types, branch markers, and ring indices (Weininger, "SMILES. 2"). However, we can't use the strings themselves as inputs for a neural network. That's why, we are going to separate it and "tokenize it". The process works this way: for each canonical SMILES $s_i = (t_{i,1}, \ldots, t_{i,L_i})$ we build a token vocabulary from the training set only, adding tokens and symbols according to the properties of the molecule. We then map each molecule to an integer sequence

$$\mathbf{z}_i = (z_{i,1}, \ldots, z_{i,L_{\max}}),$$

where tokens beyond the original length L_i are padded, and molecules longer than $L_{\max}$ are truncated at the tail. After this, a binary mask is used to indicate which positions correspond to real tokens versus padding, so that sequence models can ignore padded positions during convolution and pooling.

Graph representation. Finally, we resort to what seems the most trivial: representing molecules as graphs (atoms as nodes, and bonds as edges). Node features include one-hot encodings of atom type (element), formal charge, aromaticity flag, and optionally degree or hybridization; edge features encode bond order (single, double, triple, aromatic), conjugation, and ring membership (David et al.). Formally, each molecule x_i is mapped to

$$G_i = (V_i, E_i, \mathbf{X}_i, \mathbf{E}_i),$$

where $\mathbf{X}_i \in \mathbb{R}^{|V_i| \times d_{\text{node}}}$ and $\mathbf{E}_i \in \mathbb{R}^{|E_i| \times d_{\text{edge}}}$ collect node and edge feature vectors, collecting a lot of chemical structure and properties under a single representation.

3 Methods

3.1 Evaluation metrics and experimental setup

Given the extreme imbalance of our data (overall positive rate $\approx 0.5\%$), we will use ranking-based metrics for the evaluation of our experiments, this because accuracy fails to evaluate how good the model captures rare positives. Moreover, it is highly appropriate when only a small fraction of compounds can be tested in large-scale screening (Yuen and Franzini; Favalli et al.). For any model f_θ that outputs a real-valued score $s_i = f_\theta(x_i)$, we sort molecules in descending order of s_i and define precision and recall at rank k as

$$P(k) = \frac{\#\{i \leq k : y_i = 1\}}{k}, \qquad R(k) = \frac{\#\{i \leq k : y_i = 1\}}{\#\{i : y_i = 1\}}.$$

Average precision (AP) is the area under the precision–recall curve, written as

$$\mathrm{AP} = \sum_{k=1}^{N} P(k)\,\big(R(k) - R(k-1)\big),$$

with $R(0) = 0$.

In addition to AP, we will report the area under the ROC curve (ROC-AUC) as a secondary metric. The ROC curve can be interpreted as the probability that a randomly chosen positive receives a higher score than a randomly chosen negative. We don't give too much importance to the ROC-AUC since it can remain high even with precision at useful recall levels is poor, which is why we are mainly focusing on maximizing AP.

All models follow the block-aware splitting scheme defined in Section 2.3, with splits $\mathcal{D}_{\text{train-in}}$, $\mathcal{D}_{\text{val-ood}}$ and $\mathcal{D}_{\text{test-ood}}$ constructed from disjoint sets of DEL building blocks.[1] In all experiments:

- model parameters are learned on $\mathcal{D}_{\text{train-in}}$ only;
- hyperparameter selection and early stopping are guided by AP on $\mathcal{D}_{\text{val-ood}}$;
- final test metrics are reported on $\mathcal{D}_{\text{test-ood}}$ using the chosen configuration, with no further tuning on test data.

3.2 Tree baselines: Random Forest and XGBoost

All tree-based models will operate on the ECFP Morgan fingerprints already described in Section 2.4: each molecule x_i is mapped to a binary vector $\phi_{\text{ECFP}}(x_i) \in \{0,1\}^{2048}$ using radius-2 circular fingerprints with chirality (Rogers and Hahn; David et al.).

Random Forest

The Random Forest (RF) model is an ensemble of M decision trees $\{T_m\}_{m=1}^{M}$ trained on samples after bagging (bootstrap aggregating) of $\mathcal{D}_{\text{train-in}}$. Each leaf of the tree will produce a probability estimate for a class: $\hat{p}_m(y = 1 \mid x)$ at its leaf; the ensemble prediction is the average

$$\hat{p}_{\text{RF}}(y = 1 \mid x) = \frac{1}{M} \sum_{m=1}^{M} \hat{p}_m(y = 1 \mid x),$$

combining both bagging and the sub sampling of features randomly. This will ultimately result in a reduction of variance in the model, and a better predictive performance (Breiman). Moreover, when trained with ECFP fingerprints, the model can memorize common patterns and interactions while remaining inexpensive in comparison to other models (Kensert et al.; Cucos and Iantovics).

For our project, we use scikit-learn's `RandomForestClassifier` with log loss, bootrstrapping and oob score to monitor out of bag performance, and with aim of compensating for class imbalance we activated `class_weight = "balanced_subsample"`. We tried four different configurations for the hyperparameters:

- $M \in \{400, 800\}$ trees;
- feature subsampling `max_features` $\in$ {"sqrt", 0.2};
- depth constraint `max_depth` $\in$ {`None`, 32};
- minimum number of samples per leaf `min_samples_leaf` $\in \{1, 3, 5\}$ (paired with the above settings).

To ensure we were not just lucky with the results, each configuration is run with three random seeds $\{0, 1, 2\}$. We did all this because similar random-forest setups on molecular fingerprints yield strong baselines in virtual screening based on empirical evidence (Kensert et al.; Cucos and Iantovics).

For each configuration and seed, training proceeds in two stages:

1. **Model selection:** we fit the RF on $\mathcal{D}_{\text{train-in}}$ and compute AP and ROC-AUC on $\mathcal{D}_{\text{val-ood}}$ using the predicted probabilities on fingerprints. Here, we compare hyperparameters and seeds to identify a good baseline.
2. **Final training and test evaluation:** for the chosen configuration, we refit the model on the union $\mathcal{D}_{\text{train-in}} \cup \mathcal{D}_{\text{val-ood}}$ and report AP and ROC-AUC on $\mathcal{D}_{\text{test-ood}}$.

XGBoost

Our second tree model is gradient-boosting decision trees trained with XGBoost on ECFP fingerprints. The model maintains an additive score function

$$f_\theta(x) = \sum_{t=1}^{T} g_t(x),$$

where each g_t is a regression tree, and the probability of binding is $\hat{p}(y = 1 \mid x) = \sigma(f_\theta(x))$ with σ the logistic sigmoid. For our training objective, we will want to optimize logistic loss, while having ℓ_2 and ℓ_1 regularization on leaf weights, following the standard XGBoost formulation (Chen and Guestrin):

$$\mathcal{L}(\theta) \;=\; \sum_i \ell_{\log}(y_i, f_\theta(x_i)) \;+\; \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2,$$

where $\ell_{\log}(y, f) = -y \log \sigma(f) - (1-y) \log(1 - \sigma(f))$ and w collects the leaf parameters across all trees.

For the model configuration, we use `binary:logistic` objective with histogram-based tree construction (`tree_method = "hist"`) and `eval_metric = \aucpr`. We also set:

$$\texttt{scale_pos_weight} \;=\; \frac{N_{\text{neg}}}{N_{\text{pos}}},$$

where N_{pos} and N_{neg} are the counts of positives and negatives in $\mathcal{D}_{\text{train-in}}$.

This minimizes invariance.

The methodology afterwards mirrors what we did with Random Forests, where we used 3 different seeds around 4 different model configurations. This time we did:

[1] See Section 2.3 for the formal definition of the split and its statistics.

- maximum depth `max_depth` $\in \{6, 7, 8, 9\}$,
- learning rate $\eta \in \{0.03, 0.05, 0.10\}$,
- row subsampling `subsample` $\in \{0.8, 0.9\}$,
- column subsampling `colsample_bytree` $\in \{0.6, 0.8\}$,

with regularization parameters (λ_1, λ_2) adjusted mildly across configurations. All runs start from a high upper bound on the number of boosting rounds (`n_estimators` $= 2000$).

For each configuration and seed:

1. We train on $\mathcal{D}_{\text{train-in}}$ with an evaluation set consisting of both $\mathcal{D}_{\text{train-in}}$ and $\mathcal{D}_{\text{val-ood}}$, and use early stopping with a patience of 100 rounds. The best iteration index $t^\star$ is taken from XGBoost and we set $T_{\text{final}} = t^\star + 1$.
2. Using this early-stopped model, we compute AP and ROC-AUC on $\mathcal{D}_{\text{train-in}}$ and $\mathcal{D}_{\text{val-ood}}$.
3. We then retrain a fresh XGBoost model on $\mathcal{D}_{\text{train-in}} \cup \mathcal{D}_{\text{val-ood}}$ with `n_estimators` $= T_{\text{final}}$ fixed, and report AP and ROC-AUC on $\mathcal{D}_{\text{test-ood}}$.

3.3 Convolutional Neural Networks

For convolutional neural networks, we will work with the tokenized SMILES strings defined in Section 2.4 (Weininger, "SMILES, a Chemical Language"; Weininger, "SMILES. 2"). Each molecule x_i is represented as an integer sequence $z_i = (z_{i,1}, \ldots, z_{i,L_{\max}})$ with padding for shorter strings. An embedding layer maps tokens to vectors, yielding $E(z_i) \in \mathbb{R}^{L_{\max} \times d_{\text{emb}}}$, which is then processed by a small 1D convolutional network, similar in spirit to prior SMILES-based CNN architectures for QSAR (Hirohara et al.; David et al.).

The configuration for the neural network is as follows:

- an embedding dimension $d_{\text{emb}} = 128$;
- three parallel 1D convolutional layers with kernel sizes $(3, 5, 7)$ and $C = 256$ filters each, applied over the sequence dimension;
- a global max pooling over the sequence for each filter bank;
- a dropout layer and a final fully connected layer to a single logit.

Let $h_k \in \mathbb{R}^C$ denote the pooled output of the convolution with kernel size $k \in \{3, 5, 7\}$. We concatenate them into $h = [h_3; h_5; h_7] \in \mathbb{R}^{3C}$, apply dropout with rate 0.2, and compute a logit

$$s_i = w^\top h + b,$$

with probability of binding $\hat{p}_i = \sigma(s_i)$.

To address the severe imbalance, we train the CNN with a focal loss. For each example (x_i, y_i) with logit s_i and probability $p_i = \sigma(s_i)$, the focal loss is

$$L_i = -\alpha_t (1 - p_t)^\gamma \log(p_t),$$

where

$$p_t = \begin{cases} p_i & \text{if } y_i = 1, \\ 1 - p_i & \text{if } y_i = 0, \end{cases} \qquad \alpha_t = \begin{cases} \alpha_{\text{pos}} & \text{if } y_i = 1, \\ \alpha_{\text{neg}} & \text{if } y_i = 0, \end{cases}$$

with $\alpha_{\text{pos}} = 0.9$, $\alpha_{\text{neg}} = 0.1$ and focusing parameter $\gamma = 2.0$. Probabilities are clipped to $[10^{-6}, 1 - 10^{-6}]$ for numerical stability. The total loss is the average over a minibatch.

Adam is used to optimize (learning rate 10^{-3}, weight decay 0), using a batch size of 256 and training for up to 20 epochs. At the end of each epoch we compute AP and ROC-AUC on $\mathcal{D}_{\text{val-ood}}$; we select the checkpoint with the highest validation AP and use it to report AP and ROC-AUC on test data.

3.4 Graph neural network (GNN)

The GNN model operates on the molecular graphs described in Section 2.4. As explained, each molecule is represented as $G_i = (V_i, E_i, X_i, E_i)$, where $X_i \in \mathbb{R}^{|V_i| \times d_{\text{node}}}$ and $E_i \in \mathbb{R}^{|E_i| \times d_{\text{edge}}}$ are node and edge feature matrices (atom type, charge, aromaticity, bond order, etc.).

We employ a message-passing architecture based on GINEConv with a virtual node:

- a linear encoder maps raw node and edge features to a hidden dimension $d_{\text{hid}} = 128$;
- a stack of $L = 4$ GINEConv layers, each with its own MLP, followed by batch normalization, ReLU, and dropout with rate 0.2;
- a learned virtual node embedding $v \in \mathbb{R}^{d_{\text{hid}}}$ that is added to all node embeddings at each layer and updated via an MLP applied to the sum of node features (global context);
- a global sum pooling over nodes to obtain a graph-level embedding $h_G \in \mathbb{R}^{d_{\text{hid}}}$;
- a final MLP head mapping h_G to a single logit s_i.

At a high level, each message-passing layer updates node embeddings as

$$h_v^{(\ell+1)} = \text{MLP}^{(\ell)}\Big(h_v^{(\ell)} + \sum_{u \in \mathcal{N}(v)} \phi^{(\ell)}\big(h_u^{(\ell)}, e_{uv}\big)\Big),$$

where $\phi^{(\ell)}$ incorporates edge features, and the virtual node embedding $v^{(\ell)}$ is updated after each layer (except the last) using a pooled representation of the current node embeddings.

Due to imbalance, we will use the weighted binary cross-entropy loss with logits to regularize. Let s_i be the logit for molecule i, and $p_i = \sigma(s_i)$ the predicted probability. We compute

$$\texttt{pos_weight} = \frac{N_{\text{neg}}}{N_{\text{pos}}}$$

from $\mathcal{D}_{\text{train-in}}$ and define the per-example loss as

$$L_i = -\big(\texttt{pos_weight}\, y_i \log p_i + (1 - y_i) \log(1 - p_i)\big),$$

Table 2: OOD performance on the BRD4 split. AP is the primary model selection metric. Values are rounded to three decimals. For RF and XGB we show, for each model family, the single run with the highest AP on $\mathcal{D}_{\text{test-ood}}$.

	AP		ROC-AUC	
Model	val-ood	test-ood	val-ood	test-ood
RF (ECFP)	0.310	0.080	0.926	0.625
XGB (ECFP)	0.275	0.142	0.883	0.778
SMILES-CNN	0.394	0.329	–	0.818
GNN (PyG)	0.179	0.096	0.865	0.805

averaged over each minibatch. This will penalize the model if it misclassifies positives by too much ($N_{\text{neg}}/N_{\text{pos}}$), encouraging the network to focus on rare binders. This was based on recent GNN-based virtual screening work under scaffold-aware splits (Alves et al.; Liu et al.).

As with the CNNs, we train the GNN with Adam (learning rate 10^{-3}. However, he implement a weight decay 10^{-4}) for up to 35 epochs, using a batch size of 256 graphs. After each epoch we compute AP and ROC-AUC on $\mathcal{D}_{\text{val-ood}}$. Similarly to what we did with the CNN, we retain the checkpoint with the highest validation AP and use it to report final metrics on $\mathcal{D}_{\text{test-ood}}$.

4 Results

4.1 Performance on the OOD BRD4 split

Table 2 reports the performance of all four models on the block-based BRD4 split. We report Average Precision (AP) and ROC-AUC on both the OOD validation set $\mathcal{D}_{\text{val-ood}}$ and the OOD test set $\mathcal{D}_{\text{test-ood}}$, using AP as the primary metric (Blevins et al.; Yuen and Franzini; Favalli et al.).

For the forest models, where we used three random seeds for each, Table 2 reports the single run that achieves the highest AP on $\mathcal{D}_{\text{test-ood}}$ within that family. Due to hardware limitations, we didn't use three seeds for the Neural Network models, so we report the checkpoint with the best AP on $\mathcal{D}_{\text{val-ood}}$ and evaluate it once on $\mathcal{D}_{\text{test-ood}}$.

The clearest result is that the Convolutional Neural Network dominates, reaching AP 0.394 on $\mathcal{D}_{\text{val-ood}}$ and 0.329 on $\mathcal{D}_{\text{test-ood}}$. Among the tree models, XGB is substantially stronger than RF on the test set (AP 0.142 vs. 0.080) and achieves much higher ROC-AUC (0.778 vs. 0.625). The GNN had a test ROC-AUC 0.805, and an AP of 0.096.

When comparing the models' effectiveness in contrast with a random ranking (which would yield AP ≈ 0.015) we had the following: the CNN achieves an AP of about 0.33, corresponding to more than a twenty-fold enrichment over random, XGB reaches approx a nine-fold enrichment, the GNN about six-fold, and RF roughly five-fold.

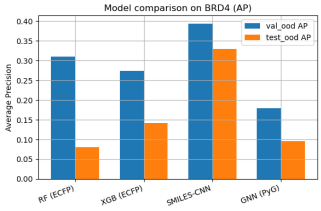


Figure 2: AP on the OOD test and val set

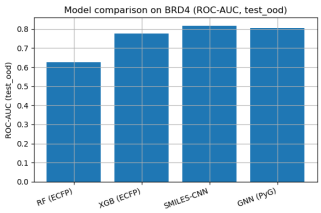


Figure 3: ROC for the 4 models

4.2 Curves and visualizations

To complement the summary metrics, we visualise ranking behaviour on the OOD test set for both AP and ROC-AUC. Figure 2 shows a model comparison on AP test and validation.

As we see, there's clearly a disparity between Random Forests and the other models in both AP and ROC-AUC. The ROC curves values highlight that the GNN achieves ROC-AUC close to the XGB and the CNN despite of its lower AP. This suggests that the GNN is able to separate classes as good as the other models, but does not focus on the probability mass on the very top-ranked candidates. The AP histogram for test/Val AP shows us that there's more separation between the validation AP of forests and the AP when tested, suggesting a degree of overfitting. On the other hand, there's not a high disparity between the Neural Networks, specially the CNN, supporting why they're widely used for virtual screening. These qualitative plots are consistent with the relative AP and ROC-AUC values reported in Table 2 and with known behaviour of RF/XGB baselines in ligand-based virtual screening (Kensert et al.; Cucos and Iantovics).

5 Discussion

5.1 Model comparison and OOD generalization

The results reveal a clear hierarchy among the evaluated model. The strongest model being the Convolutional Neural Networks, this in terms of AP and ROC-AUC in both $\mathcal{D}_{\text{val-ood}}$ and $\mathcal{D}_{\text{test-ood}}$. Then in the middle, we have XGB and GNNs, with no substantial difference between both. Finally, we can conclude that the weakest model was RF.

In absolute terms, all models work when we compare themselves to a random baseline. As stated previously, the CNN reaches AP ≈ 0.33, corresponding to more than a twenty-fold enrichment over random, while the other three models don't go above ten-fold enrichment. How-

ever, even the tiny difference between GNNs and XGB is important. These models are going to be used on virtual screening of millions of molecules, so a tiny difference would mean losing up a lot of possible candidates, which not all can be tested (Yuen and Franzini; Favalli et al.; Zhong et al.).

From a representation perspective, the fixed ECFP fingerprints combined with tree models perform well, but they struggle to match the CNN structure. As expected, RF would be poorer than XGB, since XGB is more robust. However, we may also imply that boosting can better exploit the characteristics of the ECFP structure under limited data, which could be tested in the future (Chen and Guestrin; Cucos and Iantovics).

For Neural Networks, it seems that CNNs are able to generalize really nicely the block-based OOD split when being trained on the tokenized SMILES, likely because it can capture patterns coming from the underlying chemical structure.

For GNNs, there's a high ROC-AUC, which tells us that it knows how to separate between classes, but is not fully tuned to the ranking objective by AP. This could be explained by the small amount of data in which it was trained, further research is needed to determine if they're good to generalize under large datasets. In addition, our experiments show similar trade-offs in GNN virtual screening such as the ones in Alves and Liu (Alves et al.; Liu et al.).

5.2 Limitations

This project has very important limitations:

- **No ensembling.** Each model (RF, XGB, SMILES-CNN, GNN) is evaluated separately. In practice, ensembles of diverse models normally improve accuracy (Zhong et al.). However, due to the scope of our project, we didn't try out any hybrid model or ensemble.

- **Limited data.** We would say that the greatest limitation this project has, is the data restriction. Due to the capacity of our hardware (some Intel MacBooks, which are really poor in comparison with computers with GPUs available), we had to restrict ourselves to a 50k subset, coming from a 133 million set. Larger DEL datasets, or a larger amount of data, could expose new patterns and chemical structures, which could be analyzed and captured by our models. Due to the high constraint on the size of the data, the models needed to generalize on a very small space of patterns or possible combinations, explaining why AP was not as high. Based on empirical data, a more extensive research, especially for GNNs, could yield significantly better outcomes and possibly change the ranking (David et al.; Alves et al.).

- **Single OOD split.** All the results are based on one particular block-based split. While this is highly more recommendable than random, and actually motivated by chemical work (David et al.; Yuen and Franzini), it is far away from real screening campaigns, failing to capture real difference between the testing molecules and the training domain.

5.3 Future work

Several extensions would make this project better, and could help clarify how far the conclusions we had are generalized or not to the present lack of data and CPU-only setting. Two of them are in the extent of our abilities, and would not require severe changes to the structure of the project:

More data and GPU training. First, the obvious thing to do, is to try the same models with the same configuration, under a later data set. The first step to follow, would be to test on a larger fraction of the original data set, but we could also add information coming from related DEL datasets targeting other bromodomains or epigenetic readers(Dong and Hao; Jia et al.; Dong and Wang). This will allow neural networks to fully exploit their capabilities, and may result in an even higher difference between this models and tree based models.

Alternative splitting strategies and UMAP-based OOD. Second, while the block-aware split allows us to have a OOD at the level of DEL building blocks, recent studies show that embedding molecules using ECFP, Convolutional Neural Networks, or GNN representations, reduce dimensionality with UMAP, and then perform clustering, can really yield more challenging and realistic virtual-screening experiments (gut et.al). Applying a similar UMAP clustering as the one that Guo *et al.* shows, would allow us to really determine if the models we are using are effective upon real-life drug discovery scenarios.

6 Conclusion

Our results show that the combination of extreme class imbalance and limited computational resources strongly influenced the behavior and ranking of our models. Under these constraints, the SMILES-CNN achieved the highest out-of-distribution average precision on the BRD4 subset, demonstrating that sequence-based models can extract meaningful local chemical patterns when trained on relatively small datasets. This makes the SMILES-CNN the strongest model within our computational regime.

However, the key structural insight of our study is that BRD4 activity depends on the chemical relationships encoded by the *building blocks* that form each molecule. Tree-based models do not capture these scaffold-level interactions and therefore show limited generalization under the block-aware evaluation. In contrast, the graph neural network (GNN), while not achieving the top AP score on the 50k subset, produced a high ROC-AUC and consistently learned chemically coherent graph representations of the molecules.

From these findings, two main conclusions follow:

1. **Best-performing model under our constraints.** The SMILES-CNN provides the most effective ranking of BRD4 binders in this limited-data, CPU-only setting, achieving more than a twenty-fold enrichment over random on the OOD test set.

2. **Most promising approach for scalable DEL modeling.** Given the structural nature of DEL chemistry, GNNs are best aligned with the true representation of the data and are expected to surpass other models when trained on larger fractions of BELKA or related DEL datasets.

Together, these conclusions indicate that while compact convolutional models offer strong performance in small-scale, resource-constrained environments, graph-based models represent the most suitable and scalable foundation for future BRD4 prediction efforts.

* All the code is available in a public github repository: https://github.com/apayanieu/FIT-Proteins

* On the use of AI: We used 3 AI models for this project: Chat GPT, Github Copilot, and DeepSeek. While this models guided us throughout all stages of the project, all the project's pipeline was designed by us, the final decision on the code and modeling was made by us, and all writting in this report was made by us.

Code availability All the code developed for this code is available in a public github repository: `https://github.com/apayanieu/FIT-Proteins`.

Use of AI tools We used 3 AI models for this project: Chat GPT, Github Copilot, and DeepSeek. This models guided us throughout all stages of the project, for example by suggesting code fragments, highlighting issues with both code and report, and throughout the understanding of the chemical concepts. However, all the project's pipeline, the experimental designed and model selection was designed by us. All code and analysis made by an Artificial Intelligence was reviewed and validated by us.

Works Cited

Alves, Luiz Anastacio, et al. "Graph Neural Networks as a Potential Tool in Improving Virtual Screening Programs." *Frontiers in Chemistry*, vol. 9, 2022, p. 787194.

Blevins, Andrew, et al. "NeurIPS 2024 – Predict New Medicines with BELKA." *Kaggle*, 2024, `https://www.kaggle.com/competitions/leash-BELKA`.

Breiman, Leo. "Random Forests." *Machine Learning*, vol. 45, 2001, pp. 5–32.

Chen, Tianqi, and Carlos Guestrin. "XGBoost: A Scalable Tree Boosting System." *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

Cucos, Ana-Maria, and László Barna Iantovics. "Comparative Study of Random Forest, Gradient Boosted Trees, Feedforward Neural Networks and Convolutional Neural Networks Using Fingerprints and Molecular Descriptors for Adverse Drug Reaction Prediction." *Procedia Computer Science*, vol. 246, 2024, pp. 1895–1904.

David, Laurianne, et al. "Molecular Representations in AI-Driven Drug Discovery: A Review and Practical Guide." *Journal of Cheminformatics*, vol. 12, 2020, p. 56.

Dong, Junmin, and Xiaohua Hao. "Docking-Based Virtual Screening of BRD4 (BD1) Inhibitors: Assessment of Docking Methods, Scoring Functions and *in silico* Molecular Properties." *BMC Chemistry*, vol. 18, 2024, p. 247.

Dong, Junmin, and Xinghe Wang. "Identification of Novel BRD4 Inhibitors by Pharmacophore Screening, Molecular Docking, and Molecular Dynamics Simulation." *Journal of Molecular Structure*, vol. 1274, 2023, p. 134363.

Favalli, Nicholas, et al. "DNA-Encoded Chemical Libraries: Achievements and Remaining Challenges." *FEBS Letters*, vol. 592, 2018, pp. 2168–2180.

Guo, Y., et al. "UMAP Clustering as a Robust Splitting Method for Molecular Datasets in Virtual Screening." *Journal of Cheminformatics*, vol. 17, 2025, p. 94.

Hirohara, Masashi, et al. "Convolutional Neural Network Based on SMILES Representation of Compounds for Detecting Chemical Motif." *BMC Bioinformatics*, vol. 19, 2018, p. 526.

Jia, Zhao-Tong, et al. "Identification of BRD4 (BD1) Selective Inhibitors Based on Machine Learning, Molecular Docking and Molecular Dynamics Simulation." *ChemistrySelect*, 2025.

Kensert, Alexander, Jonathan Alvarsson, and Ola Spjuth. "Evaluation of Parameters for Ligand-Based Modeling with Random Forest on Sparse Data Sets." *Journal of Cheminformatics*, vol. 10, 2018, p. 49.

Liu, Yunchao, et al. "Advancements in Ligand-Based Virtual Screening through the Synergistic Integration of Graph Neural Networks and Expert-Crafted Descriptors." *Journal of Chemical Information and Modeling*, vol. 65, 2025, pp. 4898–4905.

Rogers, David, and Mathew Hahn. "Extended-Connectivity Fingerprints." *Journal of Chemical Information and Modeling*, vol. 46, 2006, pp. 193–200.

Weininger, David. "SMILES, a Chemical Language and Information System. I. Introduction to Methodology and Encoding Rules." *Journal of Chemical Information and Computer Sciences*, vol. 28, 1988, pp. 31–36.

Weininger, David, A. Weininger, and J. L. Weininger. "SMILES. 2. Algorithm for Generation of Unique SMILES Notation." *Journal of Chemical Information and Computer Sciences*, vol. 29, 1989, pp. 97–101.

Xiao, Hang, et al. "An Invertible, Invariant Crystal Representation for Inverse Design of Solid-State Materials Using Generative Deep Learning." *Nature Communications*, vol. 14, 2023, p. 7027.

Yuen, Lik Hang, and Raphael M. Franzini. "Achievements, Challenges, and Opportunities in DNA-Encoded Library Research: An Academic Point of View." *ChemBioChem*, vol. 18, 2017, pp. 829–836.

Zhong, Feisheng, et al. "Artificial Intelligence in Drug Design." *Science China Life Sciences*, vol. 61, 2018, pp. 1191–1204.